User Stats | Question Stats | Question Details

Question Details

First Attempt: | Export to CSV | Export to Excel

Has Start Date 25/08/2025

Has End Date 01/09/2025 Apply

First Attempts (250) What do the statistics on this page

Question 1 Difficulty: 1

```
class BrewMaster {
void boilWater() { print("boil water"); }
void addPlants() { /*do nothing*/ }
void removePlants() { /*do nothing*/ }
void addCondiments() { /*do nothing*/ }
}
class BrewTea extends BrewMaster {
void addPlants() { hang tea bag }
void removePlants() { remove tea bag }
void addCondiments() { just sugar, no milk }
}
class BrewCoffee extends BrewMaster {
void addPlants() { add coffee }
//no sugar, no milk (duh)
} This is the _____ pattern
```

Answers for blank # 1

template		179	(71.6 %)
Other		71	(28.4 %)

Average Grade: 0.72 / 1 (71.
Standard Deviation: 45.09 %
Point Biserial: n/a
Discrimination Index: 49.25 %

Question 2 Difficulty: 1

```
class Editor implements IUndo {
String text; //gets saved
Window owner; //not saved
EditorSnapshot makeSnapshot(); //IUndo
void restoreSnapshot(EditorSnapshot restorePoint); //IUndo
}
class EditorSnapshot implements ISaveMe immutable {
String text;
}
interface ISaveMe immutable {
property int id;
property IUndo source;
}
interface ISaveMeList immutable { //if multiple objects must be restored
property int id;
property Datetime timestamp;
property CommandId reason;
property List<ISaveMe> sources;
void restore() { for (s in sources) { s.source.restoreSnapshot(s); }
}
class Caretaker { //manages snapshots
List<ISaveMeList> snapshots;
void add(ISaveMeList item);
void restore(int id) {
restorepoint = getById(snapshots, id);
```

```
restorepoint.restore();
```

This is the _____ pattern

Answers for blank # 1

memento	212	(84.8 %)	
Other	38	(15.2 %)	

Average Grade: 0.85 / 1 (84.8
Standard Deviation: 35.90 %
Point Biserial: n/a
Discrimination Index: 40.30 %

Question 3 Difficulty: 1

Check the pattern code in question 1. What purpose does it serve? Try and answer the question without checking your notes.

➡ Split the general steps into one class and define the details in other classes	233	(88.26 %)	
Separate commands from user interaction	6	(2.27 %)	
It's a more complex version of the command pattern	1	(0.38 %)	
All of the above	10	(3.79 %)	
No Response	14	(5.3 %)	

Average Grade: 0.93 / 1 (93.2
Standard Deviation: 25.17 %
Point Biserial: 0.24
Discrimination Index: 19.40 %

Question 4 Difficulty: 1

The pattern in question 2, implements some sort of undo functionality. How does it differ from the broken undo found in too many applications, t are based on a stack of actions that can be rolled back on at a time?

It does not, a stack of actions is exactly what it implements.	3	(1.13 %)	
➡ It saves a snapshot that includes the full state of the application at that time.	241	(90.94 %)	
It saves data to disk.	3	(1.13 %)	
It uses a FIFO list and not a LIFO stack.	2	(0.75 %)	
No Response	16	(6.04 %)	

Average Grade: 0.96 / 1 (96.4
Standard Deviation: 18.63 %
Point Biserial: 0.22
Discrimination Index: 10.45 %

Question 5 Difficulty: 1

DRY

➡ Is the first thing thought to CS students because it's so easy to spot violations.	44	(17.6 %)	
➡ Maximizes fan-in	153	(61.2 %)	
➡ Increases fan-out	18	(7.2 %)	
All code should always be DRY	109	(43.6 %)	
➡ Is not about code, but more about knowledge	132	(52.8 %)	
No Response	19	(7.6 %)	

Average Grade: 0.08 / 1 (7.5
Standard Deviation: 14.79 %
Point Biserial: n/a
Discrimination Index: 13.43 %

Question 6 Difficulty: 1

Select all below code fragments that are examples of bit parallelism (note that all code is pseudo code in an imaginary language called Q--, said language does not have weird edge cases you need worry about).

➡
```C++
a != b
```
150 (60 %)

➡
```C++
uint64 a = 1 + 2<<32; a += a;
```
175 (70 %)

```C++
int a[16]; int b[16]; /* put stuff in a, b*/
//compiler will vectorize
for (i: 0..15) { a[i] += b[i]; }
```
105 (42 %)

No Response 22 (8.8 %)

Average Grade: 0.47 / 1 (46.5
Standard Deviation: 44.11 %
Point Biserial: n/a
Discrimination Index: 67.67 %

Question 7 Difficulty: 1

Why is the acyclic visitor pattern (esp. in Java) so very very slow? Select the most correct answer.

➡ Mostly due to dynamic casts, only slightly due to indirection caused by virtual methods.	187	(70.3 %)	
Only slightly due to dynamic casts, but mostly due to indirection caused by virtual methods.	51	(19.17 %)	Average Grade: 0.75 / 1 (74.8
Not due to dynamic cast, but rather mostly due to static casts, also slightly due to virtual method calls.	4	(1.5 %)	Standard Deviation: 43.42 % Point Biserial: 0.35
Everyone knows you don't use an anti-pattern like casting in a standard pattern, it's only due to indirection (virtual calls and objects in the heap).	5	(1.88 %)	Discrimination Index: 53.73 %
No Response	19	(7.14 %)	

Question 8 Difficulty: 1

How long does the Sprint Planning Meeting take

➡ 8 hours	179	(67.55 %)	
4 hours	39	(14.72 %)	Average Grade: 0.72 / 1 (71.6
15 minutes	26	(9.81 %)	Standard Deviation: 45.09 %
It depends, somewhere between 2 weeks and 1 month	5	(1.89 %)	Point Biserial: 0.38 Discrimination Index: 62.69 %
No Response	16	(6.04 %)	

Question 9 Difficulty: 1

Why is inheritance considered an antipattern?

➡ You inherit all methods and all data from all parents, causing tight coupling.	215	(86 %)	
It's not.	13	(5.2 %)	Average Grade: 0.68 / 1 (67.6
➡ pure IS_A relationships are rare.	157	(62.8 %)	Standard Deviation: 35.07 % Point Biserial: n/a
two words: virtual methods, they suck	34	(13.6 %)	Discrimination Index: 39.55 %
No Response	18	(7.2 %)	